

ส่วนประกอบฮาร์ดแวร์	หน้าที่เชิงระบบและการวัดค่า	รูปแบบการส่งผ่านข้อมูลและอัลกอริทึมที่เกี่ยวข้อง
Teensy 4.1 Microcontroller	ประมวลผลหลักของระบบการบิน และบันทึกข้อมูลลง SD Card สำรอง	อัตราการสุ่มตัวอย่างข้อมูล 1 เอิร์ตซ์ ป้องกันการสูญหายของข้อมูล ¹
MS5611 Barometric Sensor	วัดความกดอากาศ อุณหภูมิ และคำนวณระดับความสูงทางดิ่ง	ใช้ในการคำนวณอัตราเร็วในการร่วงหล่น (Descent Rate) ¹
BNO08x Inertial Measurement Unit	วัดทัศนคติ การทรงตัว (Pitch, Roll, Yaw) และความเร่ง	ใช้ฟิเตอร์คาลมานหรือฟิเตอร์แมดกวิกในการปรับปรุงความเที่ยงตรง ¹
SAM-M10Q GPS Receiver	ระบุพิกัดภูมิศาสตร์ (Latitude,	ป้อนข้อมูลเข้าสู่อัลกอริทึมระยะ

	Longitude) และความสูงเหนือระดับน้ำทะเล	ทางฮาเวอร์ซินและพื้นที่จุดตก ¹
APC220 Wireless Transceiver	ส่งสัญญาณโทรมาตรคลื่นวิทยุ ความถี่ต่ำมายังสถานีภาคพื้นดิน (GCS)	ตรวจสอบพฤติกรรม การเคลื่อนที่ผ่านโปรแกรมตรวจสอบ พิกัด 3 มิติ ¹

ในส่วนของการนำเสนอ งานวิจัยพัฒนาจรวดระดับแข่งขัน เช่น ในโครงการ ESERO Greece's CanSat 2025 โดย Panagiotou (2025) ได้นำเสนอการออกแบบระบบจรวดนำส่งแบบนำกลับมาใช้ใหม่ได้ ซึ่งขึ้นไปถึงระดับความสูงเป้าหมาย 1,000 เมตร และทำการปล่อย CanSat ออกมาอย่างปลอดภัย.⁶ โครงการดังกล่าวประสบความสำเร็จในการบูรณาการซอฟต์แวร์จำลองวิถีการบินแบบวงปิด โดยเริ่มต้นจากการออกแบบโครงสร้างบนซอฟต์แวร์ยอดนิยมอย่าง OpenRocket และส่งผ่านพารามิเตอร์อากาศพลศาสตร์ไปวิเคราะห์ต่อด้วยแบบ 6 องศาอิสระ (6-DOF) ด้วยชุดเครื่องมือคำนวณสถิติและการแจกแจงความน่าจะเป็นบนซอฟต์แวร์ RocketPy.⁶

แบบจำลองฟิสิกส์เชิงคำนวณและพลศาสตร์การพัดปลิวตามลม

การจำลองการเคลื่อนที่ของจรวดร่วมกับ CanSat แบ่งออกเป็นสองสถานะหลักทางฟิสิกส์ ได้แก่ ช่วงการบินขึ้นแบบเร่งขับสนิ่งและลอยตัวอิสระ (Ascent Phase) ซึ่งจรวดและดาวเทียมเคลื่อนที่ร่วมกันเป็นระบบเดียว และช่วงการร่วงหล่นภายใต้ร่มชูชีพ (Descent Phase) ซึ่งทั้งสองโครงสร้างแยกตัวออกจากกันและเคลื่อนที่อย่างอิสระตามความเร็วปลายเฉพาะตัวและแรงลมภายนอก.⁶

1. กรอบอ้างอิงพิกัดเชิงตำแหน่ง (Coordinate Reference Frames)

การจำลองพิกัดการเคลื่อนที่จำเป็นต้องอ้างอิงสองระบบพิกัดหลัก ได้แก่⁸:

- **ระบบพิกัดเฉื่อยแกนโลก (Inertial Frame - IF):** วางแกนอ้างอิงคงที่กับจุดศูนย์กลางของโลก โดยแกน Z ชี้ตามแกนหมุนของโลก แกน X ชี้ไปยังพิกัดเส้นเมริเดียนกรีนิช และแกน Y ตั้งฉากตามระบบมือขวาเพื่อระบุพิกัดละติจูดและลองจิจูดเชิงมุม.⁸
- **ระบบพิกัดเคลื่อนที่ตามลำตัวจรวด (Moving Body Frame):** มีจุดกำเนิดพิกัดอยู่ที่จุดศูนย์กลางมวลของจรวด แกนสมมาตรหลักพุ่งตรงจากท้ายจรวดไปยังหัวจรวด เพื่อใช้วิเคราะห์ทิศทางของแรงขับสนิ่ง แรงยก และแรงต้านอากาศพลศาสตร์.⁸

2. สมการการแปลการเคลื่อนที่เชิงแปล (Translational Equations of Motion)

แรงภายนอกที่กระทำต่อจรวดในแนวแกนใด ๆ สามารถวิเคราะห์ได้ด้วยกฎข้อที่สองของนิวตัน ภายใต้สมมติฐานการเปลี่ยนแปลงมวลอย่างรวดเร็วเนื่องจากการเผาไหม้ของดินขับ⁸:

$$F_{total} = T + D + W = m(t)a$$

เมื่อแปลงเป็นส่วนประกอบในแนวตั้ง (ความสูง z) และแนวราบ (x, y) จะได้ความสัมพันธ์เชิงอนุพันธ์ดังนี้⁸:

$$a_z(t) = \frac{T(t)}{m(t)} - g(z) - \frac{D_z(t)}{m(t)}$$

โดยที่ $T(t)$ คือแรงขับจากมอเตอร์ ณ เวลา t , $m(t)$ คือมวลรวมของระบบที่แปรผันตามเวลาเนื่องจากอัตรา
การสิ้นเปลืองเชื้อเพลิง และ $g(z)$ คือค่าความเร่งเนื่องจากแรงโน้มถ่วงที่แปรผันตามความสูง ⁸:

$$g(z) = \frac{\mu_{\oplus}}{(R_{\oplus} + z)^2}$$

แรงต้านอากาศพลศาสตร์ (D) ขึ้นอยู่กับความหนาแน่นของอากาศ อัตราเร็ว และสัมประสิทธิ์แรงต้าน (C_d)
ของรูปทรงโครงสร้าง ¹⁵:

$$D_z = \frac{1}{2} \rho(z) V_z^2 C_d A$$

สำหรับชั้นบรรยากาศจำลองมาตรฐานสากล (International Standard Atmosphere - ISA) ความหนาแน่นของ
อากาศจะลดลงตามระดับความสูงเชิงเอ็กซ์โพเนนเชียล ⁸:

$$\rho(z) = \rho_0 e^{-\frac{z}{H}}$$

โดยที่ ρ_0 คือความหนาแน่นอากาศที่ระดับน้ำทะเล (1.225 kg/m^3) และ H คือค่าความสูงอ้างอิงเกลของ
ชั้นบรรยากาศประมาณ $8,500$ เมตร. ⁸

3. แบบจำลองกระแสลมพัดสอบเชิงลอการิทึม (Logarithmic Wind Profile)

ในการคำนวณแรงลมภายนอกที่แปรผันตามระดับความสูงจำลอง นักวิจัยใช้ความสัมพันธ์ทางชลศาสตร์ที่อธิบาย
พฤติกรรมลมในชั้นขอบเขตบรรยากาศผิวพื้น ¹⁹:

$$V_{\text{wind}}(z) = \frac{u_*}{\kappa} \ln \left(\frac{z}{z_0} \right)$$

โดยที่ $V_{\text{wind}}(z)$ คืออัตราเร็วลม ณ ระดับความสูง z เหนือพื้นดิน, u_* คือความเร็วลมเนื่องจากแรงเสียด
ทาน, κ คือค่าคงที่ฟอนคาร์มัน (≈ 0.4), และ z_0 คือระดับความขรุขระทางภูมิศาสตร์ของพื้นที่ปล่อย. ¹⁹ ระดับ
ความขรุขระนี้แปรผันตามลักษณะทางกายภาพของสภาพแวดล้อมดังแสดงในตารางเปรียบเทียบค่าคงที่ทาง
ภูมิศาสตร์ด้านล่าง ¹⁹:

ลักษณะทางภูมิศาสตร์ของพื้นที่ปล่อย	ค่าระดับความขรุขระพื้นผิว (z_0 - เมตร)	ตัวแปรประกอบการจำลองชั้นขอบเขตลม
ลานคอนกรีตเรียบหรือรันเวย์สนามบิน	0.001 –	ลมมีความเร็วสูงใกล้ผิวพื้น แรงเฉือนต่ำ

ทุ่งหญ้าโล่งหรือลานดินเรียบ	0.01 —	สภาพภูมิประเทศปกติสำหรับการแข่งขัน CanSat ⁷
พื้นที่เกษตรกรรมมีพืชคลุมดิน	0.10 —	เกิดความปั่นป่วนของลมในระดับต่ำ
พื้นที่ป่าหนาแน่นหรือปริมณฑลเมือง	0.50 —	ความเร็วลมลดลงอย่างรวดเร็วใกล้ผิวพื้น ²⁰

4. อัลกอริทึมการแปลงระยะทางและพิกัดภูมิศาสตร์ (Geographical Conversion)

เมื่อทราบระยะการพัดปลิวตามแนวราบในแกนตะวันออก-ตะวันตก (x ในหน่วยเมตร) และแนวเหนือ-ใต้ (y ในหน่วยเมตร) ที่เกิดขึ้นระหว่างการร่วงหล่น ระบบจะคำนวณตำแหน่งพิกัดภูมิศาสตร์สุดท้ายโดยแปลงระยะทางเป็นพิกัดละติจูด (ϕ) และลองจิจูด (λ) โดยอาศัยความสัมพันธ์ทางเรขาคณิตของทรงกลมโลก ¹:

$$\Delta\phi = \frac{y}{R_{\oplus}} \left(\frac{180}{\pi} \right)$$

$$\Delta\lambda = \frac{x}{R_{\oplus} \cos(\phi_{\text{start}})} \left(\frac{180}{\pi} \right)$$

$$\phi_{\text{final}} = \phi_{\text{start}} + \Delta\phi$$

$$\lambda_{\text{final}} = \lambda_{\text{start}} + \Delta\lambda$$

โดยตัวแปร $R_{\oplus}$ คือรัศมีเฉลี่ยทางดาราศาสตร์ของโลกเท่ากับ 6,378,100 เมตร. ¹⁰

สถาปัตยกรรมเชื่อมต่อและการแปลงข้อมูลจาก OpenRocket สู่ Python

การเชื่อมโยงข้อมูลจากไฟล์โครงสร้างการออกแบบของ OpenRocket (นามสกุล .ork) ไปยังโปรแกรมจำลองวิถีการบินบนภาษา Python สามารถทำได้โดยสะดวกผ่านเครื่องมือแปลงสัญญาณชุดคำสั่งของกลุ่มนักพัฒนา **RocketPy-Team** ซึ่งเปิดตัวเครื่องมือวิศวกรรมข้อมูลแบบโอเพนซอร์สชื่อว่า **RocketSerializer**. ²² ระบบซอฟต์แวร์นี้ทำงานโดยใช้อินเทอร์เฟซภาษา Java ผ่านการเรียกใช้งานโมดูลคอมไพเลอร์ JPyype เพื่อเข้าถึงแกนประมวลผลภายในของโปรแกรม OpenRocket โดยตรง. ²³ กระบวนการทำงานจะอ่านข้อมูลไฟล์ต้นทางในลักษณะโครงสร้างเอกสาร XML ของ .ork แล้วสกัดค่าคุณสมบัติกายภาพทั้งหมดออกมาให้อยู่ในรูปตัวแปรพจนานุกรม (Dictionary) ของโครงสร้างภาษา JSON หรือจัดสร้างขึ้นเป็นโครงงานสมุดบันทึกการบินสำหรับการประมวลผลบนสภาพแวดล้อมจำลอง Jupyter Notebook หรือ Google Colab โดยอัตโนมัติ. ²³

อย่างไรก็ดี การใช้งานเครื่องมือแปลรหัสอัตโนมัตินี้มีข้อจำกัดเชิงโครงสร้างที่สำคัญหลายประการ ซึ่งนักวิจัยจำเป็นต้องทราบดังนี้²³:

- **การจำกัดภาษาของฐานข้อมูล:** ไฟล์โครงสร้างการออกแบบต้นฉบับจาก OpenRocket จะต้องจัดบันทึกไว้ในสัญกรณ์รูปแบบภาษาอังกฤษเท่านั้น.²³
- **ความต้องการข้อมูลจำลองพื้นฐาน:** ตัวไฟล์ .ork จะต้องเคยผ่านการกดจำลองการประมวลผลการบิน (Run Simulation) ในโปรแกรม OpenRocket มาแล้วอย่างน้อยหนึ่งครั้งและมีประวัติผลการคำนวณบันทึกค้างไว้ในไฟล์ เพื่อให้ตัวแยกแยะทางไวยากรณ์ (Parser) สามารถดึงพารามิเตอร์จำลองไปอ้างอิงได้.²³
- **โครงสร้างทางเรขาคณิตที่จำกัด:** ตัวแปลรหัสรองรับรูปแบบจรวดชนิดขั้นตอนเดียว (Single-stage) เท่านั้น และจำกัดจำนวนการติดตั้งโครงสร้างปลายหัวจรวด (Nose Cone) เพียงชิ้นเดียว รวมถึงรองรับระบบขับเคลื่อนมอเตอร์เดียว (Single-motor Configuration) เท่านั้น.²³

ตารางการจับคู่พารามิเตอร์ XML ของ OpenRocket กับวัตถุคลาสใน RocketPy

ในการดึงข้อมูลโดยตรงจากโครงสร้าง XML ของ OpenRocket ซอฟต์แวร์จำลองภาษา Python จะทำการจำแนกแท็ก (Tag) ต่าง ๆ และจัดโครงสร้างใหม่เข้าสู่วัตถุคลาสหลักตามรูปแบบตารางข้อมูลอ้างอิงต่อไปนี้¹³:

แท็กโครงสร้าง XML ในไฟล์ .ork	ความหมายเชิงฟิสิกส์	วัตถุคลาสเป้าหมายใน RocketPy
<nosecone>	โครงสร้างปลายหัวจรวด รวมถึงวัสดุ ความยาว และความหนา	Rocket.add_nose_cone() ²⁵
<bodytube>	ลำตัวหลักของจรวดที่กำหนดพื้นที่หน้าตัดอ้างอิงเชิงพื้นที่	Rocket(radius=...) ¹³
<trapezoidalfinset>	ชุดปีกเสถียรภาพสี่เหลี่ยมคางหมู กำหนดสเปก รูทคอร์ด และทีกคอร์ด	Rocket.add_trapezoidal_fins() ²⁵
<parachute>	คุณลักษณะร่มชูชีพ รูปร่าง และแรงต้านขณะกางร่อน	Rocket.addParachute() ¹³
<launchlug>	ปุ่มนำทางรางปล่อยจรวดที่ระบุระยะเชิงมุมและตำแหน่งตามแกน	Rocket.set_rail_buttons() ²⁵
<motor>	คุณสมบัติของเครื่องยนต์ดินขับ แบ็ง ดินขับผสม หรือของเหลว	SolidMotor() / HybridMotor() ¹³

ลำดับขั้นตอนจำลองและการประมวลผลบนสภาพแวดล้อม

Google Colab

สำหรับกระบวนการพัฒนาและประมวลผลด้วยรหัสคอมพิวเตอร์ผ่านระบบ Google Colaboratory ผู้ใช้สามารถใช้บัญชีผู้ใช้งานทั่วไปเชื่อมต่อโดยตรงกับไดรฟ์ข้อมูลจำลองและเขียนคำสั่งจำลองวิธีการบินได้อย่างสะดวก รวดเร็วตามรูปแบบการทำงานสองทางเลือกดังต่อไปนี้¹⁵:

ทางเลือกที่ 1: ระบบวิเคราะห์พลศาสตร์และทำนายจุดตก 3-DOF แบบกำหนดเอง (Custom 3-DOF Simulator with Logarithmic Wind Model)

รหัสคอมพิวเตอร์ในส่วนนี้ได้รับการพัฒนาตามระบบฟิสิกส์คณิตศาสตร์บริสุทธิ์เพื่อแก้สมการการบินแปรการเคลื่อนที่ของจรวดและชิ้นส่วน CanSat โดยอิสระ โดยอ้างอิงเงื่อนไขจำลองขอยอดนิยมของการแข่งขันระดับนานาชาติ เช่น การกางร่มชูชีพสองระดับ (Dual-parachute Sequence) ของทีม Soton CanSat 2022 ซึ่งกางร่มช่วยลดความเร็วขนาดเล็กระดับสูงให้ร่วงหล่นด้วยอัตราเร็ว 15 เมตรต่อวินาที และกางร่มหลักขนาดใหญ่ที่ระดับความสูง 400 เมตร เพื่อลดอัตราความเร็วให้เหลือ 5 เมตรต่อวินาทีสำหรับความมั่นใจในการลงจอดอย่างปลอดภัย.¹²

ชุดคำสั่งนี้ถูกออกแบบมาให้รันได้ทันทีโดยไม่มีการเรียกใช้ระบบไฟล์ภายนอกที่อาจทำให้เกิดความล้มเหลวขณะประมวลผลบนคลาวด์¹⁵:

```
Python
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import

# -----
# 1. การกำหนดข้อมูลดิบแรงขับเคลื่อนและมวลของมอเตอร์ (Thrust Profile Integration)
# -----
# ข้อมูลแรงขับจำลองของเครื่องยนต์จำลอง (Estes C5-3 Motor Profile)
def get_data():
    """Time (s),Thrust (N),Mass (kg)"""
    data = [
        0.0,0.0,0.024
        0.1,5.2,0.023
        0.2,9.8,0.022
        0.3,14.5,0.020
        0.4,12.1,0.018
        0.6,6.2,0.016
        0.8,5.1,0.014
        1.0,4.8,0.012
        1.2,4.6,0.010
        1.4,4.5,0.008
        1.6,4.4,0.006
    ]
```

```
1.8,4.1,0.004
1.85,0.0,0.003
"""
```

```
# โหลดข้อมูลเข้าสู่โครงสร้าง Pandas DataFrame เหมือนทำงานบนระบบไฟล์จริง
tcurve = pd.read_csv(io.StringIO(thrust_data))

timeval = tcurve.values
thrustval = tcurve.values
massval = tcurve['Mass (kg)'].values
thrust_time = round(timeval.max, 3)
```

```
# -----
# 2. พารามิเตอร์เชิงกายภาพและตัวแปรสถานะแวดล้อม (System & Environment Constants)
# -----
g = 9.81 # ค่าแรงโน้มถ่วงเฉลี่ยของโลก (m/s^2)
rho_0 = 1.225 # ความหนาแน่นของอากาศที่ระดับน้ำทะเล (kg/m^3) [18]
dt = 0.001 # ช่วงก้าวการอินทิเกรตเวลาเชิงวินาที
```

```
# คุณลักษณะของจรวดนำส่ง (Rocket Launcher)
m_rocket_dry = 0.110 # มวลแห้งของตัวจรวด (kg)
A_rocket = np.pi * 0.02302 ** 2 # พื้นที่หน้าตัดอ้างอิงตัวจรวด (m^2)
C_d_rocket_ascent = 0.445 # สัมประสิทธิ์ต้านทานช่วงบินขึ้น
```

```
# คุณลักษณะดาวเทียม CanSat และภารกิจแยกตัว (CanSat Specifications)
m_cansat = 0.350 # มวลมาตรฐานของ CanSat (kg) [3, 18]
C_d_cansat_descent = 0.8 # สัมประสิทธิ์ต้านทานขณะอยู่ในระบบท่อ
```

```
# พารามิเตอร์พิกัดปล่อยภูมิศาสตร์ (Launch Pad Coordinates)
lat_launch = 13.7367 # พิกัดละติจูดจำลอง
lon_launch = 100.5231 # พิกัดลองจิจูดจำลอง
R_earth = 6378100.0 # รัศมีทางเรขาคณิตของโลก (m)
```

```
# -----
# 3. อัลกอริทึมจำลองแบบจำลองลมพัดสอบ (Logarithmic Wind Model Function)
# -----
def estimate_wind_profile(alt):
    """
    ประเมินทิศทางและอัตราความเร็วลม ณ ระดับความสูงต่าง ๆ ด้วยโมเดลลอการิทึม
    """
    if alt < 0.1:
        alt = 0.1

    mu_friction = 0.35 # ความเร็วลมเสียดทานเฉลี่ยผิวพื้น
    z0 = 0.02 # ระดับความขรุขระของพื้นที่ทุ่งหญ้าแข่งขัน
    kappa = 0.4 # ค่าคงที่ทางพลศาสตร์อากาศ
```

```

    speed = (u_friction / kappa) * np.log(alt / z0)

# กำหนดมุมกระแสลมเข้าปะทะหลักทำมุม 60 องศา กับแกนทิศตะวันออก
wind_angle = np.radians(60)
wind_x = speed * np.cos(wind_angle)
wind_y = speed * np.sin(wind_angle)
return wind_x, wind_y

# -----
# 4. แบบจำลองสภาพแณการจำลองวิถีขึ้น (Ascent Phase Trajectory)
# -----

time_history = 0.0
z_history = 0.0 # ระดับความสูงแนวตั้ง
vz_history = 0.0 # ความเร็วแนวตั้ง
x_drift = 0.0 # ระยะทางราบแกน X (ตะวันออก-ตะวันตก)
y_drift = 0.0 # ระยะทางราบแกน Y (เหนือ-ใต้)

t = 0.0
z = 0
m_motor_fuel = mass_val - mass_val - 1 # มวลเชื้อเพลิงเบื้องต้น
m_total = m_rocket_dry + m_cansat + mass_val # มวลรวมขณะตั้งบนแท่นปล่อย

# ค้นหาแรงขับด้วยการประมาณค่าเชิงเวลาจากข้อมูลดิบ
def get_motor_properties(current_time):
    if current_time >= thrust_time:
        return 0.0, 0.0
    # ค้นหาตำแหน่งดัชนีค่าที่ใกล้เคียงที่สุดเชิงเวลา
    idx = np.searchsorted(time_val, current_time) - 1
    if idx < 0: idx = 0
    T_step = thrust_val[idx]
    fuel_burn = (mass_val[idx] - mass_val[idx+1]) if idx < len(mass_val) - 1 else 0.0
    return T_step, fuel_burn

# ทำการเริ่มประมวลผลช่วงป้อนกำลังและช่วงลอยตัวอิสระ จนกว่าความเร็วแนวตั้งจะลดลงเป็นศูนย์
while vz_history[0] >= 0.0:
    T, fuel_loss = get_motor_properties(t)
    m_total -= fuel_loss

    # วิเคราะห์ค่าความหนาแน่นเชิงสถิติแปรผันตามระดับความสูง
    rho = rho_0 * np.exp(-z_history[0] / 8500.0)

# คำนวณแรงต้านอากาศพลศาสตร์แนวตั้ง

```



```

drag_ascent = 0.5 * rho * (vz_history[i]**2 * C_d_rock * ascent * A_rock)

# คำนวณผลความเร่งและฟังก์ชันการแปรแบบออยเลอร์ [15, 31]
az = (T / m_total) - g - (drag_ascent / m_total)

# อัปเดตฟังก์ชันความเร็วแนวราบจากอิทธิพลของลมพัดสอยภายนอก
wx,wy = estimate_wind_profile(z_history[i])

# บันทึกฟังก์ชันเสถียรทางฟิสิกส์
vz_next = vz_history[i] + az * dt
z_next = z_history[i] + vz_history[i] * dt
x_next = x_drift[i] + wx * dt
y_next = y_drift[i] + wy * dt

vz_history.append(vz_next)
z_history.append(z_next)
x_drift.append(x_next)
y_drift.append(y_next)

t += dt
time_history.append(t)
i += 1

apogee_altitude = z_history[-1]
apogee_x = x_drift[-1]
apogee_y = y_drift[-1]
print(f"--- จุดสูงสุดอะโพอิจ์ที่ประเมินได้ ---")
print(f"ระดับความสูงอะโพอิจ์: {apogee_altitude:.2f} เมตร")
print(f"ระยะพัดปลิวตามลม ณ จุดสูงสุด: X={apogee_x:.2f} ม., Y={apogee_y:.2f} ม.")

# -----
# 5. แบบจำลองพลศาสตร์ช่วงร่วงหล่นแบบสองระดับแยกชิ้นส่วน (Post-Apogee Dual Descent Phase)
# -----
# ตัวแปรร่วมสำหรับการจำลองร่วงลงของ จรวดเปล่า (Rocket Descent Core)
z_rock, x_rock, y_rock = [apogee_altitude], [apogee_x], [apogee_y]
# ตัวแปรร่วมสำหรับการจำลองร่วงลงของ CanSat (CanSat Descent Core)
z_sat, x_sat, y_sat = [apogee_altitude], [apogee_x], [apogee_y]

# กำหนดสเปกของร่มชูชีพในการลงจอด
# จรวดนำส่งใช้ร่มชูชีพเดียวเพื่อนำกลับมาใช้ใหม่
A_rock_chute = np.pi * 0.12 ** 2
C_d_rock_chute = 1.35

# ดาวเทียม CanSat ใช้การประมวลผลทางร่มแบบสองระดับ (Dual-parachute Stage Sequence)

```

```
# ระยะแรกเหนือกุด 400 เมตร จะใช้ร่มเล็ก (Drogue) อัตราการร่วงหล่น ~15 m/s
```

```
A_sat_drogue = np.pi * 0.04 * 2
```

```
C_d_sat_drogue = 1.1
```

```
# ระยะสองต่ำกว่า 400 เมตร จะกางร่มชูชีพหลัก (Main Chute) อัตราการร่วงหล่น ~5 m/s
```

```
A_sat_main = np.pi * 0.14 * 2 * 15
```

```
C_d_sat_main = 1.75 * 15
```

```
# จำลองจรวดเปล่าร่อนลงกระทบพื้นดิน
```

```
idx = 0
```

```
while z_rock[-1] > 0.0
```

```
    curr_alt = z_rock[-1]
```

```
    rho = rho_0 * np.exp(-curr_alt / 8500.0)
```

```
# อัตราความเร็วปลายคงที่ขณะร่วงหล่นของร่มจรวด [18]
```

```
v_terminal_rock = np.sqrt(2 * m_rocket_dry * g / (rho * C_d_rock_chute
```

```
A_rock_chute))
```

```
wx, wy = estimate_wind_profile(curr_alt)
```

```
z_rock.append(curr_alt - v_terminal_rock * dt)
```

```
x_rock.append(x_rock[-1] + wx * dt)
```

```
y_rock.append(y_rock[-1] + wy * dt)
```

```
# จำลอง CanSat ร่อนลงกระทบพื้นดินแบบสองระดับเชิงเงื่อนไข
```

```
while z_sat[-1] > 0.0
```

```
    curr_alt = z_sat[-1]
```

```
    rho = rho_0 * np.exp(-curr_alt / 8500.0)
```

```
# การคำนวณแบ่งเฟสของร่มตามความสูงจริง (Trigger Logic)
```

```
if curr_alt >= 400.0
```

```
    # เฟสที่ 1: ร่มนำร่องประกอบเสถียรภาพ (Drogue Phase)
```

```
    v_terminal_sat = np.sqrt(2 * m_cansat * g / (rho * C_d_sat_drogue * A_sat_drogue))
```

```
else
```

```
    # เฟสที่ 2: ร่มชูชีพหลักกางทำงานเพื่อลงจอดอย่างปลอดภัย (Main Phase)
```

```
    v_terminal_sat = np.sqrt(2 * m_cansat * g / (rho * C_d_sat_main * A_sat_main))
```

```
wx, wy = estimate_wind_profile(curr_alt)
```

```
z_sat.append(curr_alt - v_terminal_sat * dt)
```

```
x_sat.append(x_sat[-1] + wx * dt)
```

```
y_sat.append(y_sat[-1] + wy * dt)
```

```

# -----
# 6. รายงานตำแหน่งและการแปลงพิกัดพิกัดเชิงแผนที่ (Final GPS Location Reporting)
# -----

delta_lat_rock = (y_rock[1] / R_earth) * 180 / np.pi
delta_lon_rock = (x_rock[1] / (R_earth * np.cos(np.radians(lat_launch)))) * 180 / np.pi

delta_lat_sat = (y_sat[1] / R_earth) * 180 / np.pi
delta_lon_sat = (x_sat[1] / (R_earth * np.cos(np.radians(lat_launch)))) * 180 / np.pi

lat_final_rock = lat_launch + delta_lat_rock
lon_final_rock = lon_launch + delta_lon_rock

lat_final_sat = lat_launch + delta_lat_sat
lon_final_sat = lon_launch + delta_lon_sat

print(f"\n--- การคำนวณตำแหน่งลงจอดสุดท้ายตามพิกัดภูมิศาสตร์ ---")
print(f"ตำแหน่งจรวดนำส่งตกลงบนพื้น: Lat {lat_final_rock:.6f}, Lon {lon_final_rock:.6f}")
print(f"ตำแหน่งดาวเทียม CanSat ตกลงบนพื้น: Lat {lat_final_sat:.6f}, Lon {lon_final_sat:.6f}")

# -----
# 7. วาดกราฟเปรียบเทียบผลเพื่อประเมินความปลอดภัย (Visual Verification Output)
# -----

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(x_drift, z_history, 'g-', label='Ascent Phase')
plt.plot(x_rock, z_rock, 'r--', label='Rocket Descent')
plt.plot(x_sat, z_sat, 'b--', label='CanSat Descent')
plt.xlabel('Horizontal Drift Distance [m]')
plt.ylabel('Altitude [m]')
plt.title('Flight Path 2D Side View')
plt.grid(True)
plt.legend()

plt.subplot(1, 2, 2)
plt.scatter(0, 0, color='orange', marker='*', s=150, label='Launch Pad')
plt.plot(x_rock, y_rock, 'r-', alpha=0.6, label='Rocket Track')
plt.plot(x_sat, y_sat, 'b-', alpha=0.6, label='CanSat Track')
plt.scatter(x_rock[1], y_rock[1], color='red', marker='x', s=100, label='Rocket Impact')
plt.scatter(x_sat[1], y_sat[1], color='blue', marker='o', s=100, label='CanSat Impact')
plt.xlabel('East-West Drift [m]')
plt.ylabel('North-South Drift [m]')
plt.title('Surface Landing Trajectory footprint')
plt.grid(True)

```

```
plt.legend()
plt.tight_layout()
plt.show()
```

ทางเลือกที่ 2: ระบบจำลองวิถี 6-DOF และการวิเคราะห์การกระจายตัวด้วย RocketPy (Professional RocketPy Integration Platform)

การสร้างสคริปต์สำหรับการทดสอบสถิติเชิงปริมาณของพิกัดจุดตกผ่านชุดไลบรารีวิทยาศาสตร์ระดับมาตรฐานสากล สามารถดำเนินแนวทางการเขียนคำสั่งตามรูปแบบจำลองดังต่อไปนี้¹⁶:

Python

```
# ขั้นตอนการติดตั้งชุดไลบรารีคำนวณพลิกศาสตร์อากาศยาน [27]
!pip install rocketpy netCDF4 contextily -c
```

```
from rocketpy import Environment, SolidMotor, Rocket, Flight
import datetime
```

```
# 1. การประกาศพิกัดและโมเดลชั้นบรรยากาศอ้างอิง (Environmental Formulation) [26, 28]
```

```
env = Environment()
latitude = 32.990254
longitude = 106.974998
elevation = 1400 # ความสูงของพื้นที่เหนือระดับน้ำทะเล (m)
```

```
# การเข้าถึงฐานข้อมูลทำนายลมตามเวลาพยากรณ์ผ่านอินเทอร์เน็ต GFS [26, 28]
```

```
tomorrow = datetime.date.today() + datetime.timedelta(days = 1)
env.set_date((tomorrow.year, tomorrow.month, tomorrow.day), 12 # ปรับตั้งค่าเวลา UTC
env.set_atmospheric_model(type = "Forecast" file = "GFS"
```

```
# 2. จัดสร้างวัตถุมอเตอร์ระบุอัตราเร่งขับเคลื่อน (Motor Modeling) [13, 27]
```

```
# พารามิเตอร์นี้สกัดได้จากการอิมพอร์ตโมเดลเครื่องยนต์ของ OpenRocket ผ่าน RocketSerializer
cesaroni_m1670 = SolidMotor()
```

```
thrust_source = "https://raw.githubusercontent.com/RocketPy-Team/RocketPy/master/data/motors/cesaroni/Cesaroni_M1670.eng" # แหล่งข้อมูลโปรไฟล์เครื่องยนต์ตัวอย่าง [32]
```

```
dry_mass = 1.815
propellant_initial_mass = 2.5
nozzle_radius = 0.033
```

```

nozzle_throat_radius= 0.011
grain_number=5
grain_density=1815
grain_outer_radius=0.033
grain_inner_radius= 0.015
grain_height 0.12
grain_spacing 0.005
combustion_chamber_to_nozzle=0
|

```

```

# 3. ประกอบโครงสร้างตัวจรวดเสถียรภาพสูง (Dynamic Vehicle Assembler) [27, 28]
custom_rocket = Rocket
radius= 127 2000 # รัศมีกายภาพของลำตัวจรวด (m)
mass=14.426 # น้ำหนักมวลแห้งของจรวดนำส่งไม่รวมอุปกรณ์ดินขับและ CanSat
inertia= 6.321 6.321 0.034 # ความเฉื่อยทางฟิสิกส์ (Roll-Pitch Inertia Tensors)
power_off_drag 0.42 # สัมประสิทธิ์ต้านทานอากาศเมื่อเครื่องยนต์ดับ
power_on_drag= 0.45 # สัมประสิทธิ์ต้านทานอากาศเมื่อเครื่องยนต์ติดเครื่อง
center_of_mass_without_motor=0
|

```

```

# กำหนดสัณฐานงานสำหรับตรวจสอบความเร็วหลุดร่วง [26, 28]
custom_rocket.set_rail_buttons
upper_button_position= 0.0818
lower_button_position= 0.618
angular_position=45
|

```

```

# ติดตั้งมอเตอร์ขับเคลื่อนเข้ากับตำแหน่งท้ายระบบ [28]
custom_rocket.add_motor(cesaroni_m1670, position= 1.255)

```

```

# การเชื่อมต่อชิ้นส่วนความปลอดภัยเสถียรภาพทางอากาศพลศาสตร์ [27, 28]
custom_rocket.add_nose_cone
group_name="Ogive Nose Cone"
length= 0.274
kind="ogive"
position= 1.134
|

```

```

custom_rocket.add_trapezoidal_fins
n= 4
root_chord= 0.120
tip_chord= 0.060
span= 0.110

```

```

position=-1.04956
|

# 4. ประกาศกลไกการร่มชูชีพนำร่องและหลัก (Parachute Ejection Mechanics) [29]
def drogue_ejection_trigger(p, y):
    # สั่งการเปิดร่มชูชีพนำร่องทันทีที่อัตราความเร็วแนวตั้งเป็นลบ (Apogee Detected)
    return True if <0 else False

def main_ejection_trigger(p, y):
    # สั่งการเปิดร่มชูชีพหลักเมื่อระดับความสูงอยู่ต่ำกว่า 400 เมตร เหนือระดับพื้นดิน
    return True if <0 and y < 400 < 1400 else False

custom_rocket.addParachute(
    "Drogue Chute"
    CdS=1.15
    trigger=drogue_ejection_trigger
    samplingRate=105
    lag=1.5
    noise=0 8.3 0.5
|

custom_rocket.addParachute(
    "Main Chute"
    CdS=11.2
    trigger=main_ejection_trigger
    samplingRate=105
    lag=1.5
    noise=0 8.3 0.5
|

# 5. ประมวลผลแบบจำลองเส้นทางบินเชิง 6-DOF (Flight Trajectory Solver Execution) [27]
flight_simulator = Flight
rocket=custom_rocket
environment=env
rail_length=5.2
inclination=85
heading=0
|

# พิมพ์ผลรายงานตัวแปรวิกฤตทางกายภาพของเที่ยวบิน [26, 27]
flight_simulator.info()

```

การวิเคราะห์ความเปี่ยงเบนเชิงสถิติมอนเตคาร์โลและวิวัฒนาการ

ระบบควบคุมทิศทาง

การทำนายวิถีจุดตกของ CanSat มีวิวัฒนาการที่ก้าวหน้าไปกว่าการทำนายตำแหน่งบนจุดอ้างอิงเชิงเดี่ยว (Deterministic Prediction) เนื่องจากกระแสลมภายนอก สภาวะอุณหภูมิจึงและมวลรวมของระบบจริงมีความคลาดเคลื่อนเชิงฟิสิกส์อยู่เสมอ.⁸ การประยุกต์ใช้ **ระเบียบวิธีมอนเตคาร์โล (Monte Carlo Simulation)** จึงเป็นมาตรฐานสากลที่ใช้ในการคำนวณการกระจายตัวของพิกัดจุดตก.⁸

กระบวนการนี้ทำงานโดยสร้างแบบจำลองการจำลองการบินซ้ำจำนวน 1,000 ถึง 10,000 รอบ โดยกำหนดการสุ่มกระจายตัวแปรนำเข้าที่สำคัญแบบสุ่มที่มีการแจกแจงแบบปกติ (Gaussian Distribution) เช่น กำหนดความ

ผันผวนของมวลแห้งที่ผิดพลาดได้ $\pm 5\%$, อัตราส่วนความเร็วลมพัดสอบเพิ่มลดได้ $\pm 10\%$, และระดับความ

ล่าช้าในการเปิดทางของร่มชูชีพ (Trigger Lag) ในช่วง ± 0.5 วินาที.²⁹ ผลลัพธ์สุดท้ายจะปรากฏเป็นกลุ่มจุดตกบนแผนที่ภูมิศาสตร์ ซึ่งช่วยให้นักวิจัยสามารถวาดรูปวงรีความเชื่อมั่น (Landing Ellipse) ภายในขอบเขตรัศมี

ความเชื่อมั่นสถิติระดับ 3σ (ความน่าจะเป็นครอบคลุมร้อยละ 99.7) สำหรับควบคุมรัศมีความปลอดภัยและการจำกัดขอบเขตพื้นที่ค้นหาและกู้ภัยทางภูมิศาสตร์ได้อย่างดีเยี่ยม.⁸

นอกเหนือจากการจำลองสถิติแล้ว ในการออกแบบ CanSat ขั้นสูง นักวิจัยยังประยุกต์ใช้เทคโนโลยีการร่อนจอดอย่างแม่นยำ (Autonomous Guided Landing) เพื่อควบคุมทิศทางอย่างกระตือรือร้น (Active Controls).¹¹ โดยสถาปัตยกรรม CanSat แบบพาราพอยล์นำทาง (Steerable Parachute System) จะได้รับการติดตั้งชุดขับเคลื่อนมอเตอร์เซอร์โว (Servo Motors) ทำหน้าที่ดึงสายบังคับร่มชูชีพซ้ายและขวาเพื่อปรับเปลี่ยนรูปทรงของร่ม ส่งผลให้เกิดแรงบังคับเชิงเสถียรและการเล็งเป้าเข้าสู่พิกัดนำร่องที่กำหนดล่วงหน้า.⁴

การทำงานร่วมกันระหว่างเซ็นเซอร์กรองสัญญาณเชิงทัศนคติของแมดกวิก (Madgwick Filter IMU Integration) และการส่งค่าพิกัดตรวจวัดเปรียบเทียบระยะทางภูมิศาสตร์ผ่านอัลกอริทึมระยะทางฮาเวอร์ซิน จะช่วยปรับค่าสัญญาณความกว้างพัลส์ (PWM) ของมอเตอร์เซอร์โว เพื่อดึงสายควบคุมร่มชูชีพพา CanSat เคลื่อนที่เลี้ยวซ้าย ขวา หรือกดหน้าตัวร่มเพื่อลดแรงต้านอากาศ เพิ่มประสิทธิภาพการเจาะกระแสลมพัดเฉือน และลงจอดในขอบเขตรัศมีความแม่นยำต่ำกว่า 10 เมตรจากจุดศูนย์กลางเป้าหมายได้อย่างปลอดภัยสูงสุด.⁴

สรุปและแนวทางการใช้งานเชิงปฏิบัติ

การผสานสถาปัตยกรรมโปรแกรมจำลองวิถีการบิน OpenRocket เข้าสู่ระบบการประมวลผลคำนวณทางคณิตศาสตร์บนสภาพแวดล้อม Python ถือเป็นขั้นตอนที่จำเป็นในการวิจัยทางวิศวกรรมการบินระดับมาตรฐานสากล.⁸ การนำกระบวนการและทฤษฎีต่าง ๆ ไปประยุกต์ใช้งานจริงอย่างมีประสิทธิภาพ มีข้อพิจารณาเชิงเทคนิคที่สำคัญดังนี้:

- **การรวบรวมข้อมูลกายภาพที่สมบูรณ์:** ก่อนทำการอัปโหลดวีดิโอให้ไฟล์ .ork ผ่านชุดคำสั่ง RocketSerializer หรือ orhelper นักวิจัยต้องตรวจสอบให้แน่ใจว่าได้ทำการระบุค่าความเฉื่อยเชิงฟิสิกส์ (Inertia Tensors) และค่าพิกัดตำแหน่งอ้างอิงศูนย์กลางถ่วง (Center of Gravity) ที่ถูกต้องตามมาตรฐานกายภาพของอุปกรณ์จริงลงในโมเดลโครงสร้างแล้ว เพื่อป้องกันความคลาดเคลื่อนสะสมขณะคำนวณความเสถียรทางการบินเชิงมุม (Dynamic Stability Analysis).¹⁴
- **ความสำคัญเชิงสถิติของกระแสลมระดับพาดสูง:** กระแสลมพัดสอบ (Wind Shear) มีความแปรปรวนเชิงทิศทางและกำลังตามระดับความสูงของชั้นขอบเขตบรรยากาศ การจำลองการบินที่ถือเอาแรงลมภายนอกเป็นค่าคงที่สม่ำเสมอตลอดเวลาจะนำไปสู่ความคลาดเคลื่อนสะสมในการพยากรณ์พิกัดจุดลงจอดอย่างรุนแรง การนำแบบจำลองลอการิทึมผิวพื้นหรือการเชื่อมต่องานพยากรณ์ลมแบบเวลาจริง GFS มาประมวลผล จึงเป็นสิ่งจำเป็นอย่างยิ่งในการเพิ่มความเชื่อถือได้ของการทำนายพิกัดกู้ชีพ.⁶
- **ระบบความปลอดภัยทางวิศวกรรม:** นักวิจัยควรตั้งค่านิยามการล้มแบบสองระดับเชิงเงื่อนไข

(Dual-parachute Sequences) เป็นค่าเริ่มต้นในการจำลองเสมอ เนื่องจากแรงเฉื่อยของ CanSat และความเร็วปลายในการปะทะพื้นดิน เป็นปัจจัยวิกฤตที่ขัดขวางการหยุดของอุปกรณ์ฮาร์ดแวร์เซ็นเซอร์ที่ละเอียดอ่อนจากการกระแทกในตอนท้ายสุด.¹²

ผลงานที่อ้างอิง

1. Real-time Telemetry, Flight Path 3D Visualization, CanSat Attitude, and Landing Location Prediction - ResearchGate, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://www.researchgate.net/publication/399101221_Real-time_Telemetry_Flight_Path_3D_Visualization_CanSat_Attitude_and_Landing_Location_Prediction
2. Research Article Experimental Approach for the Evaluation of the Performance of a Satellite Module in the CanSat Form Factor for - Semantic Scholar, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://pdfs.semanticscholar.org/1ab8/f579d7ff102affa9e8cb9fb38e7290274acf.pdf>
3. CanSat - Grokipedia, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://grokipedia.com/page/CanSat>
4. CanSat 2021 - Herotech, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://herotech.ro/cansat.html>
5. APOGEE PREDICTION 1 A Machine Learning-Based Apogee Prediction Methodology for Experimental Student Rockets Price Hamilton Drawd - Liberty University, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://digitalcommons.liberty.edu/cgi/viewcontent.cgi?article=2672&context=honors>
6. Design, Fabrication and Launching of CanSat-Deploying High-Power Rockets - MDPI, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://www.mdpi.com/2673-4591/133/1/84>
7. DESIGN CHALLENGES, AND OUTCOMES OF BUILDING A SATELLITE THE SIZE OF A SODA CAN - UPCommons, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://upcommons.upc.edu/bitstreams/5fa35656-49c1-4b0a-bf12-9e503b64aa3a/download>
8. Development of a Model Rocket Trajectory Simulation Tool with Python - ResearchGate, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://www.researchgate.net/publication/368490407_Development_of_a_Model_Rocket_Trajectory_Simulation_Tool_with_Python
9. (PDF) nthroot Calculator - ResearchGate, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://www.researchgate.net/publication/391533330_nthroot_Calculator
10. GUI built in PyQt (Python) for a Can-Sized Sattelite (CanSat) - GitHub, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://github.com/AnshTanwar/GUI-Cansat-22>
11. Constructing a Can Sized Sat With Steering - CanSat 2025 : 7 Steps - Instructables, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://www.instructables.com/Constructing-a-Can-Sized-Sat-With-Steering-CanSat-/>
12. CanSat 2022 Preliminary Design Review (PDR) Outline Version 1.1, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://cansatcompetition.com/docs/teams/cansat2022_1032_pdr_v1.pdf
13. Getting Started - RocketPy in Colab, เข้าถึงเมื่อ มิถุนายน 9, 2026

- https://docs.rocketpy.org/en/rel-v1.0.0a1/notebooks/getting_started.html
14. Source code for rocketpy.rocket.rocket, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://docs.rocketpy.org/en/latest/modules/rocketpy/rocket/rocket.html>
 15. Python Model Rocket Trajectory Simulator Tutorial - SpaceLab Illinois, เข้าถึงเมื่อ มิถุนายน 9, 2026
https://learnrockets.spacelab.web.illinois.edu/uploads/Python_Code_Rocket_Trajectory_Simulator_Final.pdf
 16. GitHub - RocketPy-Team/RocketPy: Next generation High-Power Rocketry 6-DOF Trajectory Simulation, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://github.com/RocketPy-Team/RocketPy>
 17. Rocket Simulation by using Machine learning - Kaggle, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://www.kaggle.com/code/rajeevsinghisodiya/rocket-simulation-by-using-machine-learning>
 18. CanSat Report - ESERO Luxembourg, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://www.esero.lu/wp-content/uploads/2026/02/Final-CanSat-Report-SevenToHeaven-.pdf>
 19. Fitting a two-layer model to wind profile data in python - Stack Overflow, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://stackoverflow.com/questions/35795341/fitting-a-two-layer-model-to-wind-profile-data-in-python>
 20. Logarithmic wind profile doesn't give expected results - Forums, Autodesk, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://forums.autodesk.com/t5/cfd-forum/logarithmic-wind-profile-doesn-t-give-expected-results/td-p/7891538>
 21. Simulation and Analysis of Atmospheric Boundary Layer Characteristics in a Wind Tunnel Using Python - Engineered Science Publisher, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://www.espublisher.com/journals/articledetails/2148/>
 22. GitHub - openrocket/openrocket: Model-rocketry aerodynamics and trajectory simulation software, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://github.com/openrocket/openrocket>
 23. RocketPy-Team/RocketSerializer: Easily convert your OpenRocket files into RocketPy simulations. - GitHub, เข้าถึงเมื่อ มิถุนายน 9, 2026
<https://github.com/RocketPy-Team/RocketSerializer>
 24. Scripting with Python and JPyype - OpenRocket wiki, เข้าถึงเมื่อ มิถุนายน 9, 2026
http://wiki.openrocket.info/Scripting_with_Python_and_JPyype
 25. File Specification — OpenRocket 23.09 documentation, เข้าถึงเมื่อ มิถุนายน 9, 2026
https://openrocket.readthedocs.io/en/latest/dev_guide/file_specification.html
 26. First Simulation with RocketPy, เข้าถึงเมื่อ มิถุนายน 9, 2026
https://docs.rocketpy.org/en/latest/user/first_simulation.html
 27. Getting Started - RocketPy in Colab, เข้าถึงเมื่อ มิถุนายน 9, 2026
https://docs.rocketpy.org/en/latest/notebooks/getting_started_colab.html
 28. Getting Started — RocketPy 1.12.1 documentation, เข้าถึงเมื่อ มิถุนายน 9, 2026
https://docs.rocketpy.org/en/latest/notebooks/getting_started.html
 29. Getting Started — RocketPy 0.13.1 documentation, เข้าถึงเมื่อ มิถุนายน 9, 2026
https://docs.rocketpy.org/en/v0.13.1/notebooks/getting_started.html

30. Team Sammard's Official GCS Software for CanSat 2022 - GitHub, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://github.com/Jatin7385/CanSat_2022_GCS
31. 6-DOF Atmospheric Rocket Landing Guidance using Meta-Reinforcement Learning - electronic library -, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://elib.dlr.de/210766/1/Jacopo_Carradori_Thesis_reduced.pdf
32. Monte Carlo simulations — RocketPy 1.0.0a1 documentation, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://docs.rocketpy.org/en/rel-v1.0.0a1/notebooks/dispersion_analysis/dispersion_analysis.html
33. Monte Carlo Simulations — RocketPy 1.12.1 documentation, เข้าถึงเมื่อ มิถุนายน 9, 2026 https://docs.rocketpy.org/en/latest/notebooks/monte_carlo_analysis/monte_carlo_analysis.html
34. Design and Navigation Control of an Advanced Level CANSAT, เข้าถึงเมื่อ มิถุนายน 9, 2026 <https://cltp.info/pdf/paper4.pdf>